

Ros Day03 hw1 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.11

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 ROS2 Day03 과제에서는 Day01 에서 제작한 turtlesim 도형 그리기 패키지에 Parameter 를 추가하고, 선의 색상과 굵기를 YAML 파일에서 설정하도록 프로그램을 수정하였다.

처음에는 Parameter, YAML, Launch 가 각각 어떤 역할을 하는지 정확하게 이해하기 어려웠다. 특히 YAML 파일에 값을 작성한 후 해당 값이 소스 코드까지 어떻게 전달되는지와 Launch 파일이 필요한 이유를 이해하는 데 어려움이 있었다.

따라서 AI 에게 Day03 강의자료의 내용을 설명해 달라고 요청하였다. 이전에 학습한 Day01 과 Day02 내용도 함께 참고하여 이번 과제와 어떤 연관성이 있는지 질문하였다.

또한 AI 에게 코드 작성에 대하여 질문하였다. AI 의 답변은 코드를 작성하는데 도움을 받았다.

2. AI 사용 부분

2.1 강의 자료 이해

AI 에게 Day03 강의자료의 내용을 설명하고 이전 강의자료와 연결해 달라고 질문하였다. AI 는 Day01 부터 Day03 까지의 내용을 다음과 같은 흐름으로 설명하였다.

Day01에서는 Node 와 Topic 통신을 학습하고 turtlesim 을 이용하여 도형을 그리는 프로그램을 제작하였다.

Day02에서는 Qt GUI 와 ROS2 통신을 학습하였다.

Day03에서는 기존에 제작한 여러 Node 를 Launch 파일로 실행하고, 프로그램에서 사용하는 설정값을 Parameter 와 YAML 파일로 관리한다

내가 지금 ros가 뭔지 모르겠거든 자세하게 내가 보낸 자료를 바탕으로 추가자료가 필요하면 검색해도 되고 아무튼 나에게 좀 설명해줘 기본적으로 pdf에 적혀있는 내용들을 중심으로

16m 37s 동안 작업 >

좋아요. 보내주신 자료에서 말하는 것은 정확히는 **ROS 2**입니다. 아래에서는 편의상 “ROS”라고도 부르지만 전부 ROS 2를 뜻합니다.

1. ROS 2가 대체 무엇인가?

ROS는 **Robot Operating System**의 약자입니다. 하지만 Windows나 Ubuntu처럼 컴퓨터를 직접 운영하는 운영체제는 아닙니다.

정확히는 다음을 제공하는 **로봇 소프트웨어 개발 프레임워크이자 도구 생태계**입니다.

- 여러 프로그램이 데이터를 주고받는 통신 기능
- 센서·제어·화면 프로그램을 따로 만들어 연결하는 구조
- 다른 사람이 만든 로봇 패키지를 재사용하는 방법
- 프로그램을 빌드하고 실행하는 도구
- 여러 프로그램을 한 번에 실행하는 Launch
- 실행 중인 프로그램과 통신 상태를 조사하는 명령어

실제 운영체제는 Ubuntu 같은 Linux이고, ROS 2는 그 위에서 동작합니다.

로봇 하나를 회사라고 생각하면 쉽습니다.

ROS 개념	회사에 비유하면
노드(Node)	카메라 담당자, 판단 담당자, 모터 담당자
메시지(Message)	정해진 양식으로 작성한 문서
토픽(Topic)	계속 정보가 올라오는 방송 채널
서비스(Service)	질문 하나를 보내고 답을 받는 문의 창구
액션(Action)	오래 걸리는 일(↓)이고 진행 상황을 받는 작업 주문

5. Day 3: Parameter, YAML, Launch

Day 3의 핵심은 다음 한 줄입니다.

코드에는 행동을 쓰고, YAML에는 바꿀 값을 쓰고, Launch에는 무엇을 함께 실행할지 쓴다.

.bashrc 와 alias

PDF의 `cw`, `cs`, `cb`, `sb` 등은 긴 명령을 줄이는 단축어입니다.

예:

```
</> Bash
alias cw='cd ~/colcon_ws'
alias cs='cd ~/colcon_ws/src'
alias cb='cd ~/colcon_ws && colcon build'
alias sb='source ~/colcon_ws/install/setup.bash'
```

이것들은 ROS의 핵심 기능이 아니라 작업 편의를 위한 셸 설정입니다.

자료에는 `gb`가 두 번 선언돼 있어 뒤의 설정이 앞의 설정을 덮어씁니다. 또한 `killg`의 `rosmaster`는 ROS 1 개념입니다. ROS 2는 ROS 1처럼 중앙 `rosmaster`를 사용하지 않습니다.

ROS_DOMAIN_ID

같은 네트워크에서 여러 사람이 ROS 2를 실행하면 서로의 노드가 발견될 수 있습니다. 이를 논리적으로 분리하는 값이 `ROS_DOMAIN_ID`입니다.

```
</> Bash
export ROS_DOMAIN_ID=10
```

- 서로 통신해야 하는 노드: 같은 ID
- 다른 사람의 실습과 분리: 서로 다른 ID

이 값은 보안 암호가 아니라 통신 그룹을 나누는 번호입니다. = [ROS_DOMAIN_ID 공식 설명](#)

YAML



2.2 코드 작성 및 변형

AI에게 부탁하여 기존 패키지안, 소스코드에 있던 선의 굵기와 색상을 YAML 파일에 옮기는 역할을 수행하였다.

46s 동안 처리함

ㅇㅋ. 지금 코드 구조 봤으니까 **최소 수정으로 과제 1 끝내자**. 네 기존 도형 그리는 로직은 그대로 두고, `setPen(255, 0, 0, 3)` 처럼 코드에 박혀 있던 색/굵기를 YAML에서 가져오게만 바꾸면 된다. 과제 요구사항도 “굵기, 색상을 파라미터 설정하고 yaml 파일에서 불러오기”다.

1. YAML 파일부터 만들기

현재 `parkdoyeon_day3_hw1` 폴더니까 그대로:

```
</> Bash
mkdir -p config
nano config/params.yaml
```

내용:

```
</> YAML
turtle_drawer:
  ros__parameters:
    pen_r: 255
    pen_g: 0
    pen_b: 0
    pen_width: 3
```

현재 설정은 **빨간색 + 굵기 3**.

저장:

```
</> 일반 텍스트
Ctrl + O
Enter
Ctrl + X
```

2. 헤더에 변수 4개 추가

3. 깨달은 점

이번 과제를 통해 Parameter 는 단순한 일반 변수가 아니라 Node 의 동작을 외부에서 설정하기 위한 값이라는 것을 알게 되었다.

기존에는 선의 색상과 굵기를 변경하려면 소스 코드의 값을 직접 수정해야 했다. 하지만 Parameter 와 YAML 을 적용한 후에는 프로그램의 주요 코드를 수정하지 않고 YAML 파일의 값만 변경하여 다른 결과를 확인할 수 있다는 것을 알게 되었다.

특히 YAML 파일에 값을 작성하는 것만으로 프로그램이 변경되는 것이 아니라, 소스 코드에서 같은 이름의 Parameter 를 선언하고 읽는 과정이 필요하다는 점을 알게 되었다.

Launch 파일을 사용하면 turtlesim Node 와 직접 제작한 Node 를 각각 다른 터미널에서 실행하지 않고 하나의 명령으로 실행할 수 있다는 점도 알게 되었다. 프로그램을 실행할 때마다 같은 명령을 반복해서 입력할 필요가 없다는 점도 깨달았다.

이번 과제를 통해 Parameter, YAML 및 Launch 의 기본적인 사용 목적과 연결 과정을 이해할 수 있었다. 또한 AI 는 과제를 대신 수행하는 용도보다 이해하지 못한 개념을 질문하고 전체적인 구현 방향을 정리하는 용도로 사용하는 것이 더 도움이 된다고 느꼈다.